

INF-2700: Database systems

Assignment 3

Jens Christian Valen Leynse

jeley4465

Nov 06th, 2023

Introduction

The assignment is separated into 3 parts. Part 1 is implementing a natural join operation with both a tuple- and a block-based design. This part assumes that the two tables only has a single common integer attribute. The next part is about the performance of the implementation. By making the program handle large tables the objective is to profile and compare the performance of the two algorithms. Lastly a written-only part is included where the goal is to do a thought experiment comparing the block nested-loop join algorithm with an algorithm that makes use of the B⁺-tree file organizations.

Design & Implementation

Task 1: Implementing natural join

The natural join function aims to merge two tables into a single table, combining fields with identical data. It begins by creating a new schema and allows users to choose between two join algorithms: a tuple-based or a block-based algorithm. The choice of the algorithm is mainly intended for debugging and performance assessment.

The heart of the natural join function involves finding and combining common fields from both input schemas. These common fields serve as the basis for merging records. To start, a new schema is created to hold the combined data. The algorithm then identifies common fields and adds them to the new schema, excluding duplicates.

In cases where no common field is found between the two schemas, a message is displayed, indicating the absence of a suitable field for the join operation.

For scenarios where a common field is identified, the algorithm checks if it's an integer field. Depending on the specified key (0 for tuple-based or 1 for block-based), the appropriate method for joining records is chosen.

Tuple-Based Join:

- This method iterates through the pages of both schemas, searching for matching values in the common field. Records with matching values are merged and added to the resulting schema. The loop continues until all records are processed.

Block-Based Join:

- In this approach, the algorithm iterates through the blocks of both the left and right schemas. For each block, it further iterates through the pages. The algorithm extracts records based on the common field's offset, compares the values, and merges matching records. When a matching value is found, the records are combined and appended to the resulting schema. Each page is unpinned after processing.

The 'merging records' function is responsible for combining records. It copies the input schemas and initializes field pointers and counters. The counters help maintain the current field positions in all three schemas. The function loops through the fields in the resulting schema, checking for matching field names in the source schemas and adding data accordingly. Once all fields are processed, the temporary schemas are removed since they are no longer needed.

In summary, the natural join function creates a new schema to merge two tables. It finds common fields and uses either a tuple-based or block-based method to combine records, depending on the key provided. The 'merging records' function efficiently combines the data, and the resulting schema represents the merged table. If no common field is found, the function reports the absence of a suitable field for the join operation.

Task 3: Performance & Discussion

Unfortunately for the sake of profiling the performance it's impossible with the current code configuration. The process can be ran giving the correct resulting table. But a segmentation fault interrupts the test code, stopping it before it can get meaningful data to compare. What little data it is able to display, is the same for both join methods regardless of the number of records. Similarly with the front code, the program is able to use the provided table creation and query to natural join both tables. However, it fails to detect the end of the records, making it unable to process both join methods for the comparison of performance.

Regrettably, due to the current code configuration, profiling the performance is impossible. The process can run successfully, generating the expected resulting table. However, it is interrupted by a segmentation fault in the test execution code. This interruption occurs before meaningful data can be collected for comparison. The limited data that is available indicates that both join methods perform similarly, regardless of the number of records. Similarly, in the front execution code, the program can utilize the provided table creation and query to perform a natural join on both tables. Nevertheless, it fails to detect the end of the records, rendering it incapable of processing both join methods for performance comparison.

Task 4: Think out of the box

Another join algorithm that can leverage the B+-tree file organization system is the sort-merge join algorithm. This algorithm relies on the sorted presentation of both input datasets, which naturally aligns with the organization of B+-trees.

The sort-merge join algorithm works by first sorting both input datasets by the join attribute. This sorting process ensures that the algorithm can perform linear scans of both datasets in an interleaved manner, efficiently matching records with the same join attribute values. In essence, the sort-merge join can be thought of as an inner join of two relations based on a single attribute.

In terms of I/O complexity, the block nested-loop join has a worst-case complexity of either $O(P_r \cdot P_s / M)$ I/Os, where P_s and P_r represent the sizes of the join operands in pages, and M

is the available pages of internal memory, or $O(P_r + P_s)$ I/Os if the outer join operand fits in available internal memory.

On the other hand, the sort-merge join algorithm exhibits a similar $O(P_r + P_s)$ I/O complexity in the worst case. However, it's important to note that if the input datasets are not already sorted by the join attribute, an additional sorting step is necessary. This sorting step introduces an $O(P_r \log(P_r) + P_s \log(P_s))$ I/O complexity, which can impact the overall performance.

In summary, both algorithms have the potential to benefit from B+-tree file organization. The choice between them depends on various factors, including the availability of memory, the organization of the input data, and the desired trade-offs between sorting and joining. The sort-merge join is advantageous when datasets are already sorted, as it can achieve similar I/O complexity with less memory usage compared to the block nested-loop join.

Conclusion

The program successfully performs natural joins to merge tables, with the nested loop join algorithm including all records. Unfortunately, performance profiling is unfeasible due to technical issues, preventing a comparison of join methods. In the "think out of the box" task, the sort-merge join algorithm is introduced, highlighting its benefits with B+-tree file organization. This assignment covers designing a natural join function, tackling issues in measuring performance, and proposing a creative sort-merge join method. Although the design is well-built, there are technical constraints that make it challenging to measure performance. The assignment provides valuable insights into natural joins and improving database management systems.